

Tutorato 1: Teoria e Basi dell'Assembly RISC-V

1 Domande a risposta multipla (Teoria)

Esercizio 1

Quanti registri generali possiede l'architettura base del RISC-V a 64 bit e qual è la loro dimensione?

- (A) 16 registri da 32 bit
- (B) 32 registri da 64 bit
- (C) 64 registri da 32 bit
- (D) 32 registri da 32 bit

Esercizio 2

Qual è lo scopo principale del registro `x0` nell'architettura RISC-V?

- (A) Viene utilizzato come Program Counter (PC).
- (B) Punta alla cima dello stack (Stack Pointer).
- (C) Contiene sempre il valore costante 0 e non può essere modificato.
- (D) Contiene l'indirizzo di ritorno per le chiamate a procedura.

Esercizio 3

Quale formato di istruzione viene utilizzato in RISC-V per eseguire operazioni che richiedono un operando costante (ad esempio l'istruzione `addi`)?

- (A) Formato R (Registro)
- (B) Formato I (Immediato)
- (C) Formato S (Store)
- (D) Formato J (Jump)

Esercizio 4

Considerando l'ordine di memorizzazione dei byte in memoria (endianness), quale convenzione utilizza l'architettura RISC-V vista nel corso?

- (A) Big Endian (il byte più significativo all'indirizzo più basso)
- (B) Little Endian (il byte meno significativo all'indirizzo più basso)
- (C) Middle Endian
- (D) L'ordine dei byte è specificato dal programmatore tramite uno pseudo-codice.

2 Traduzione in Assembly RISC-V

Esercizio 5

Tradurre la seguente espressione C in Assembly RISC-V. Si assuma che le variabili `a`, `b`, `c`, `d` siano assegnate rispettivamente ai registri `x10`, `x11`, `x12`, `x13`. Utilizzare un registro temporaneo a scelta per i passaggi intermedi.

```
a = (b + c) - d;
```

Esercizio 6

Tradurre la seguente istruzione di accesso in memoria ad un array in Assembly RISC-V. Si assuma che l'indirizzo base dell'array di *double word* `A` sia contenuto in `x22` e che la variabile `h` sia in `x21`.

```
A[4] = h + A[2];
```

Esercizio 7

Tradurre la seguente operazione C sfruttando in modo efficiente le operazioni logiche del processore RISC-V (senza usare l'istruzione `mul`). Si assuma che `a` sia in `x10` e `b` sia in `x11`.

```
a = b * 8;
```

Esercizio 8

Tradurre il seguente blocco condizionale `if-else` in assembly RISC-V. Si assuma che le variabili `i`, `j`, `f`, `g`, `h` siano assegnate rispettivamente ai registri `x22`, `x23`, `x19`, `x20`, `x21`.

```
if (i < j)
    f = g + h;
else
    f = g - h;
```

Esercizio 9

Tradurre il seguente semplice costrutto `if` in assembly RISC-V. Si assuma che le variabili `i` e `j` siano assegnate ai registri `x22` e `x23`, e la variabile `f` sia in `x19`.

```
if (i == j) {
    f = f + 1;
}
```

Esercizio 10

Tradurre il seguente blocco `if-else` con un controllo di minoranza. Si assuma che `i` sia in `x22`, `j` in `x23`, `f` in `x19`, `g` in `x20` e `h` in `x21`.

```
if (i < j)
    f = g;
else
    f = h;
```

Esercizio 11

Tradurre la seguente operazione condizionale che usa una disuguaglianza. Si assuma che le variabili `i` e `j` siano assegnate ai registri `x22` e `x23`.

```
if (i != j) {
    i = i + 5;
}
```

Esercizio 12

Tradurre il seguente codice che esegue un calcolo e poi un semplice controllo sul risultato. Assumere che `a` sia in `x10`, `b` in `x11` e `c` in `x12`.

```
a = b + c;
if (a < 0) {
    a = 0;
}
```

Esercizio 13

Tradurre il seguente ciclo `while` in assembly RISC-V. Si assuma che l'indice `i` sia nel registro `x22`, la variabile `k` in `x24` e l'indirizzo base dell'array `salva` sia in `x25`. L'array è composto da *double word*.

```
while (salva[i] == k)
    i += 1;
```